Locality versus Balance in Finite-Capacity Edge Caches

Author names to be finalized
Affiliations and contact details to be supplied by the authors

***Abstract*—Routing requests to a stable hash owner preserves cache locality, while spreading requests can improve request-count balance but duplicate fills across finite-capacity edge caches. We evaluate this trade-off in MiniCDN, a single-host Docker prototype, using four seeded synthetic workloads and 10-repetition, 3,000-request-per-trial sweep compares consistent hashing, round robin, and adaptive spillover. Under Zipf demand, adaptive cache-hit ratio rose from 74.74% at slack 0 to 90.87% at slack 1000 while excess origin attempts fell from 523.5 to 39.8 per trial; request-count coefficient of variation rose from 0.084 to 0.313. In the repeated-key cold-burst trace, consistent hashing and slack 1000 averaged one origin attempt, whereas round robin and adaptive slack 0–8 averaged three. Paired intervals show round robin's Zipf 95th-percentile latency exceeds consistent hashing by 32.88 ms (95% confidence interval [8.18, 57.59]), while adaptive-sweep comparisons do not support a latency ranking. At slack 2, the 95th-percentile latency of spilled cache misses was not consistently higher than that of non-spilled cache misses. One completed uniform/slack-1000 condition contained 19 errors; a later same-session baseline attempt also failed and is excluded. These results describe a closed-loop, single-host synthetic setup, not production CDN performance.**

***Index Terms*—content delivery networks, consistent hashing, edge computing, caching, load balancing**

I. Introduction

An edge cache's routing policy determines which requests share a cache. Stable key ownership tends to concentrate each object's copies, preserving effective capacity. A request-balanced policy can instead send the same key to different edges, increasing duplicate fills and eviction pressure. Under finite capacity, the policy trades off locality, balance, origin work, and latency.

We ask: *How does request-load-aware spillover change cache locality, origin work, request-count balance, and client latency under finite edge-cache capacity?* This work evaluates an experimental platform rather than proposing a new hashing theorem or a production-ready CDN design. Contributions include an instrumented benchmark; a repeated slack sweep with a distinct-key lower bound on origin work; and a comparison with two explicitly router-local load heuristics.

The results are mixed. For uniform and Zipf workloads, higher slack improves HIT ratio and decreases excess origin attempts while increasing request-count CV. The highest slack behaves much like consistent hashing on Zipf, while several intermediate settings remain behind it. Cold-burst traces favor keeping repeated keys together. Spill-latency results vary by workload and cache status and do not establish a universal cold-edge penalty.

II. Related Work

Consistent hashing supports stable object assignment as server membership changes; its foundational work also studies random cache trees for hot-spot relief [1]. Consistent hashing with bounded loads formalizes a balance/movement trade-off for client-to-server assignments [2]. These allocation-count guarantees do not directly imply request-latency guarantees under nonuniform popularity or unequal service costs. Later work addresses cascaded overflow with a minwise-hashing approach [3]; our router-local heuristic is not an implementation of either algorithm.

Locality-Aware Request Distribution (LARD) combines request-content affinity with backend load to increase cache locality while balancing a cluster [4]. LARD is the closest conceptual predecessor to our locality/load trade-off. MiniCDN does not dynamically partition a working set: it begins with a consistent-hash owner and redirects only when router-local in-flight counts cross a slack threshold. The power-of-two-choices literature studies load reduction by selecting among two candidate queues [5]; MiniCDN instead compares the hash owner with the least-loaded available ring candidate, so it is not a power-of-two-choices implementation.

Production systems also combine hashing and load distribution at different layers. Maglev is Google's software network load balancer, using consistent hashing and connection tracking, and its authors report its use for Google traffic [6]. Nygren et al. describe the architecture and operation of Akamai's distributed content-delivery platform [7]. Maggs and Sitaraman connect algorithmic work to practical CDN subsystems and deployment constraints [8]. These systems establish operational context, but do not provide directly comparable MiniCDN cache-routing measurements.

CDN surveys describe the broader system components, replication approaches, request-redirection mechanisms, and evaluation methods [9], [10]. Dynamic replica placement studies where content copies should reside to scale delivery [11]; this paper instead measures how a router selects among already-running finite-capacity edge caches.

Cache-aware load balancing has been deployed for data-center search applications, where a weighted voting scheme combines term locality with replica selection [12]. MiniCDN routes one HTTP resource key per request and uses synthetic traces, so the mechanism and evaluation are not directly comparable. Cache stampede prevention addresses concurrent regeneration of expired objects [13]; MiniCDN fill coalescing

is a distinct mechanism. HTTP conditional request precedence follows RFC 9110 [14]. HAProxy's configuration manual documents least-connection balancing and 'hash-balance-factor', which enables bounded-load consistent hashing [15]. These are production-software reference points; the MiniCDN bounded-load router-local policy remains a separate in-flight-count heuristic and is not an implementation of HAProxy's algorithm or the Mirrokni et al. object-placement algorithm.

III. System and Routing Policies

MiniCDN comprises a router, three edge caches, an origin, and a Redis-backed control service, run using Docker Compose. The edges use bounded body-byte capacity and LRU replacement. Each experiment configures 64 KiB body capacity per edge and 1 KiB origin objects; metadata and transient buffers are excluded.

The primary evaluated modes are consistent hash (route to hash owner), round robin, and adaptive spillover. Adaptive routing preserves the hash owner unless its router-local in-flight reservation count exceeds the least-loaded candidate by more than configured slack. The sweep uses slack values 0, 1, 2, 4, 8, and 1000; the final value approximates pure hash for these runs. Completed least-loaded and bounded-load router-local measurements came from a separate Compose session and are excluded from primary comparisons; a later same-session run did not complete. The origin handler follows RFC 9110 conditional-request precedence and is covered by regression tests.

IV. Experimental Method

A. Configuration

The main sweep uses uniform, Zipf (exponent 1.1), changing-hot-set, and repeated-key cold-burst traces. Each trial has 256 warm-up requests, followed by an acknowledged cache purge and 3,000 measured requests. Seeds derive deterministically from base seed 2100260 plus run index. Each condition has ten repetitions, concurrency is 16, cache TTL is one hour, and origin delay is 20 ms. The environment is a single Docker host with closed-loop clients and no network impairment.

The primary same-session sweep contains consistent hashing, round robin, and six adaptive slack values across four workloads (320 trials total). Two additional router-local policies were measured in another Compose session (80 trials), but those results are excluded from primary comparisons to avoid a session confound. A later attempted interleaved comparison stopped at 68 of 200 trials and is not analyzed as a completed experiment.

B. Metrics and Statistics

HIT ratio is the fraction of requests reported as cache hits. For each trial, the compulsory-miss floor is the number of distinct requested keys after the cache reset; excess origin attempts equal actual origin attempts minus this floor. Request-count coefficient of variation (CV) is computed over the three edge counts. Per-request instrumentation records trial ID, key, serving edge, hash owner, spill status, `X-Cache`, and client latency.

HIT ratio, origin attempts, distinct-key floor, excess attempts, and request CV use two-sided 95% Student t-intervals over independent trial-level values. P50/P95/P99 latencies and error counts use deterministic 10,000-resample percentile bootstrap intervals for the mean (seed 3102026). Latency percentiles and spill groups are reduced per trial before inference; individual requests are not treated as independent replicates.

V. Results

A. Retained Comparison

The retained October comparison is reproduced from `routing-comparison.json` and `routing-requests.jsonl`. Its Zipf HIT rates are 80.34% (consistent hash), 65.49% (round robin), and 76.69% (adaptive); mean Zipf origin attempts are 151, 265, and 179. Uniform HIT rates are 60.89%, 21.09%, and 48.44%. Mean distinct-key floors are 151.00 (Zipf), 176.33 (changing hot), 242.67 (uniform), and 1.00 (cold burst). Derived values are in `routing-reproduction-2026-10-25.json` and `routing-trials-derived-2026-10-25.csv`.

B. Same-Session Policy and Slack Results

In the extended sweep, uniform adaptive HIT ratio increases from 30.99% at slack 0 to 72.66% at slack 1000, while excess attempts fall from 1814.2 to 567.0 per trial. For Zipf, HIT increases from 74.74% to 90.87%, and excess attempts fall from 523.5 to 39.8. Zipf request CV rises from 0.084 to 0.313. At slack 1000, Zipf closely matches consistent hashing: 90.88% HIT, 39.4 excess attempts, and CV 0.313. These results and their intervals are derived from `routing-sweep-2026-10-25T19-23-50-602Z.json` and `routing-analysis-primary-2026-11-03-summary.csv`.

For Zipf at slack 2, adaptive records 83.70% HIT, 254.9 excess attempts, CV 0.184, and mean P95 76.6 ms. Consistent hash records 90.88%, 39.4, 0.313, and 73.3 ms. Round robin records 73.02%, 575.3, 0.000, and 106.2 ms. These are descriptive means across ten trials, not paired inferential comparisons. The Zipf P95 confidence intervals overlap for consistent hash and adaptive slack 0–8; no latency ranking is claimed. The slack-1000 P95 interval also overlaps consistent hashing, so similar HIT and excess-origin results do not establish a latency advantage.

In cold burst, the per-trial distinct-key floor is one. Consistent hash and adaptive slack 1000 average one origin attempt and 99.97% HIT. Round robin and every adaptive setting from slack 0 through 8 average three origin attempts and 99.90% HIT. Slack 1000 has CV 1.414, showing that preserving one repeated key can produce uneven request counts.

One condition has nonzero errors: uniform adaptive slack 1000 recorded 19 errors among 30,000 requests. Its mean P95 is 176.2 ms with bootstrap 95% interval [62.4, 384.9] ms. This condition is interpreted alongside the errors and not as a reliable performance advantage. The trial summaries are in `routing-analysis-primary-2026-11-03-`

summary.csv. A same-seed 10-trial replay had zero errors in 30,000 requests; its source is routing-sweep-2026-10-26T05-10-19-985Z.json, and it does not erase the original error observation.

C. Capacity-Only Check

At 64 KiB per edge with 1 KiB objects, each edge holds 64 objects. For 256 uniformly requested keys, a capacity-only prediction is 64/256 = 25% HIT for round robin and (3×64)/256 = 75% for consistent hash. Measured round-robin HIT is 25.05% (95% t-interval [24.35, 25.75]); consistent hash is 73.49% ([73.15, 73.83]). The prediction falls within the round-robin interval and above the consistent-hash interval. This descriptive comparison does not identify the source of the residual. Measured values are from routing-analysis-primary-2026-11-03-summary.csv; configuration is in routing-sweep-2026-10-25T19-23-50-602Z.json.

D. Paired Differences

The primary sweep uses the same deterministic trace for each workload/run across policies. We computed per-run differences versus consistent hashing and two-sided 95% paired Student t-intervals over ten run-level pairs. HIT differences are percentage points; other metrics use their original scales. The full table and figure are routing-analysis-primary-2026-11-03-paired.csv and ../results/figures/routing-analysis-primary-2026-11-03-paired-differences.pdf. Condition order was not randomized, so pairing does not remove possible time/order effects. The intervals are unadjusted for multiple comparisons and should be treated as exploratory.

On Zipf, round robin's P95 difference versus consistent hashing is +32.88 ms (95% CI [+8.18, +57.59]); its HIT ratio is 17.86 percentage points lower. For every adaptive slack setting, the paired P95 interval includes zero, so no adaptive latency ranking is claimed. At slack 1000, paired Zipf HIT and excess-origin differences also include zero.

E. Spill Latency

For adaptive slack 2, spilled MISS P95 is higher than non-spilled MISS P95 for cold burst (39.0 vs. 35.0 ms), but lower for changing hot (91.0 vs. 99.7 ms), uniform (92.0 vs. 93.1 ms), and Zipf (126.5 vs. 128.7 ms). The results do not support a general claim that spilled requests explain adaptive's higher tail latency. Spill status is observational, not a randomized causal treatment. Trial-level confidence intervals are in routing-analysis-primary-2026-11-03-spill-summary.csv; the source request export is routing-sweep-2026-10-25T19-23-50-602Z-requests.csv.

At adaptive slack 2, the MISS share among classified spilled requests is 92.10% for uniform demand versus 39.03% for non-spilled requests. Across all adaptive trials, pooled OLS of spill fraction against HIT ratio has $R^2 = 0.0023$; against excess origin attempts, $R^2 = 0.0216$. Residual RMSEs are 17.851 percentage points and 469.415 attempts per trial, respectively. This pooled fit has little explanatory power and is descriptive, not causal. The regression summary/residuals, spill fractions, and per-workload MISS rates are in routing-analysis-primary-2026-11-03-spill-regression-summary.csv, routing-analysis-primary-2026-11-03-spill-regression-trials.csv, routing-analysis-primary-2026-11-03-spill-fraction-trials.csv, and routing-analysis-primary-2026-11-03-spill-miss-rates.csv. Rates are computed across trial-level proportions; request rows without a spill flag and HIT/MISS classification are excluded.

VI. Discussion

Increasing slack improves reuse and decreases excess origin work in uniform and Zipf traces, but it also increases request-count CV. The evidence does not establish that an adaptive load-aware policy outperforms consistent hashing overall. Round robin balances request counts but pays more duplicate origin work. Router-local baseline measurements are exploratory only and are omitted from primary comparisons because they came from a separate session.

Spill-latency results reject the initial simple explanation of a universal cold-edge penalty. Because spill status was not randomized, this is observational evidence and not a causal estimate.

VII. Threats to Validity

The experiment uses one Docker host, a closed-loop client, synthetic traces, three edge instances, and a synthetic fixed origin delay. It does not establish behavior under open-loop arrivals, WAN latency, heterogeneous edges, production request distributions, large objects, shared multi-router load state, or changing membership. Four workload generators and one Zipf exponent limit generalization. One complete condition contains 19 errors; its count and P95 are disclosed, and the recorded evidence does not establish the cause. A same-seed replay had no errors but does not erase the original observation. A later interleaved sweep stopped at 68 of 200 trials with 1,563 HTTP 502/503 failures after routers logged response-header timeouts and unhealthy-ring responses; the underlying trigger is undetermined, and the partial run is excluded from outcome comparisons. Earlier router-local measurements came from a separate Compose session and are not used as primary baseline evidence. Ten trial-level replicates per condition are not a substitute for independent hosts. The distinct-key floor assumes a cold-start trace and does not model prewarmed or externally shared caches. Cache capacity counts body bytes but excludes metadata and runtime overhead.

VIII. Conclusion

The measured balance/locality trade-off is workload-dependent. In uniform and Zipf traces, higher adaptive slack

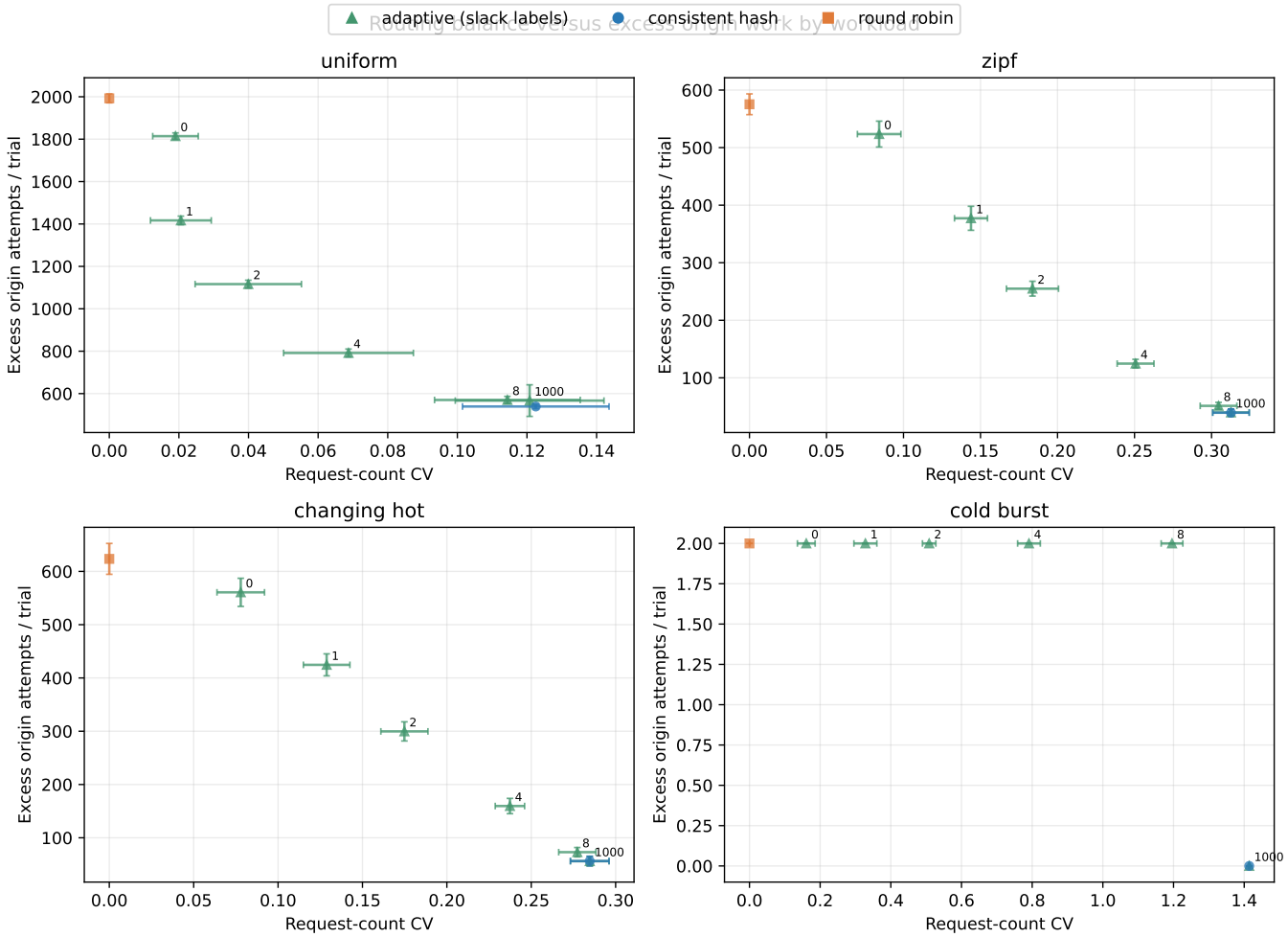


Fig. 1. Excess origin attempts versus request-count CV. Points summarize same-session policy/workload conditions; error bars are trial-level 95% t-intervals. Source: `routing-sweep-2026-10-25T19-23-50-602Z.json`.

moves toward hash locality, improving HIT ratio and reducing excess origin attempts while increasing request-count imbalance. Round robin balances requests but incurs more duplicate origin work. Consistent hashing is strong on locality-sensitive workloads, and slack 1000 approximates it on Zipf. A repeated-key cold burst shows spillover can turn one compulsory fill into multiple origin attempts. Per-request evidence does not show spilled requests are universally slower, so high P95 must not be attributed to spillover without controlled causal experiments.

Future work should evaluate cache-size, origin-delay, and Zipf-exponent sensitivity; repeat experiments on independent hosts with open-loop arrivals; investigate the error-bearing condition; and compare with a faithful bounded-load consistent-hashing implementation.

Data Availability

All numerical results are derived from the dated JSON/CSV files under `docs/academic/results/`. The retained source files are `routing-comparison.json` and `routing-requests.jsonl`. The primary analysis uses `routing-sweep-2026-10-25T19-23-50-602Z.json` and its request export; the separate-session router-local file and incomplete 68/200 run are excluded. Analysis is generated with `scripts/analyze-routing-sweep.py`; trial execution uses `scripts/compare-routing.mjs`. The command log and remaining tasks are in `REPORT.md`.

References

[1] D. Karger, E. Lehman, T. Leighton, M. Levine, D. Lewin, and R. Panigrahy, "Consistent hashing and random trees: Distributed caching protocols for relieving hot spots on the world wide web," in *Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing*, 1997, pp. 654–663.

[2] V. Mirrokni, M. Thorup, and M. Zadimoghaddam, "Consistent hashing with bounded loads," in *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms*, 2018, pp. 587–604.

[3] J. Chen, B. Coleman, and A. Shrivastava, "Revisiting consistent hashing with bounded loads," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 5, pp. 3976–3983, 2021.

[4] V. S. Pai, M. Aron, G. Banga, M. Svendsen, P. Druschel, W. Zwaenepoel, and E. M. Nahum, "Locality-aware request distribution in cluster-based network servers," in *Proceedings of the Eighth International Conference*

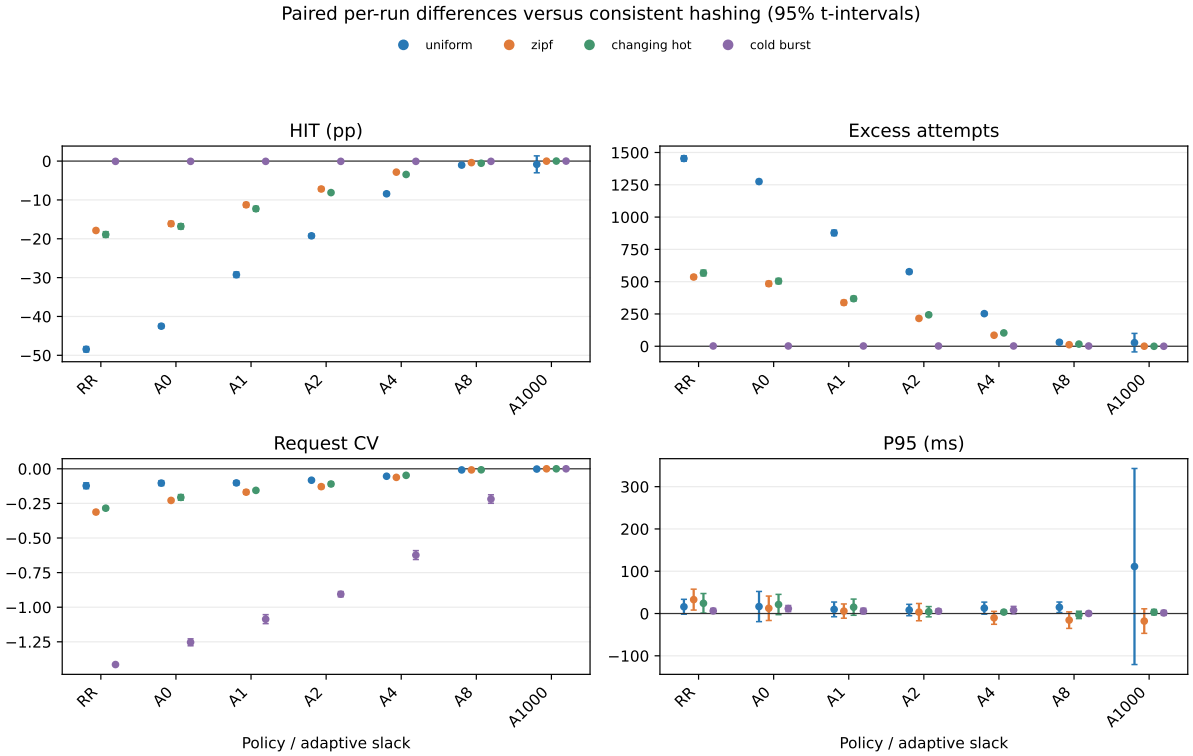


Fig. 2. Paired run-level differences from consistent hashing across four workloads. Positive/negative values indicate larger/smaller outcomes for round robin or adaptive routing; bars are 95% paired t-intervals. Source: `routing-analysis-primary-2026-11-03-paired.csv`.

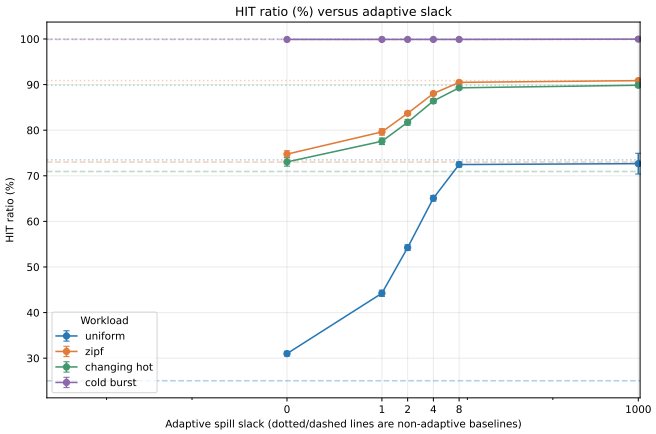


Fig. 3. HIT ratio versus adaptive spill slack. The plot includes non-adaptive consistent-hash and round-robin reference lines. Source: `routing-sweep-2026-10-25T19-23-50-602Z.json`.

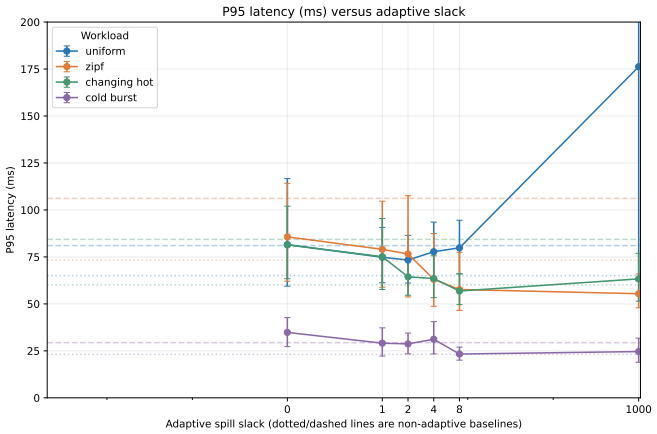


Fig. 4. P95 client latency versus adaptive spill slack. Error bars are trial-level bootstrap 95% intervals for the mean. The y-axis is capped at 200 ms; the annotation identifies the 19-error condition whose interval exceeds the displayed range. Source: `routing-sweep-2026-10-25T19-23-50-602Z.json`.

on Architectural Support for Programming Languages and Operating Systems, 1998, pp. 205–216.

[5] M. Mitzenmacher, "The power of two choices in randomized load balancing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 12, no. 10, pp. 1094–1104, 2001.

[6] D. E. Eisenbud, C. Yi, C. Contavalli, C. Smith, R. Kononov, E. Mann-Hielscher, A. Cilingiroglu, B. Cheyney, W. Shang, and J. D. Hosein, "Maglev: A fast and reliable software network load balancer," in *Proceedings of the 13th USENIX Symposium on Networked Systems Design and Implementation*. USENIX Association, 2016, pp. 523–535. [Online]. Available: https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/eisenbud

[7] E. Nygren, R. K. Sitaraman, and J. Sun, "The akamai network," *ACM SIGOPS Operating Systems Review*, vol. 44, no. 3, pp. 2–19, 2010.

[8] B. M. Maggs and R. K. Sitaraman, "Algorithmic nuggets in content delivery," *ACM SIGCOMM Computer Communication Review*, vol. 45, no. 3, pp. 52–66, 2015.

[9] A. Vakali and G. Pallis, "Content delivery networks: Status and trends," *IEEE Internet Computing*, vol. 7, no. 6, pp. 68–74, 2003.

[10] J. Sahoo, M. A. Salahuddin, R. Glitho, H. Elbiaze, and W. Ajib, "A

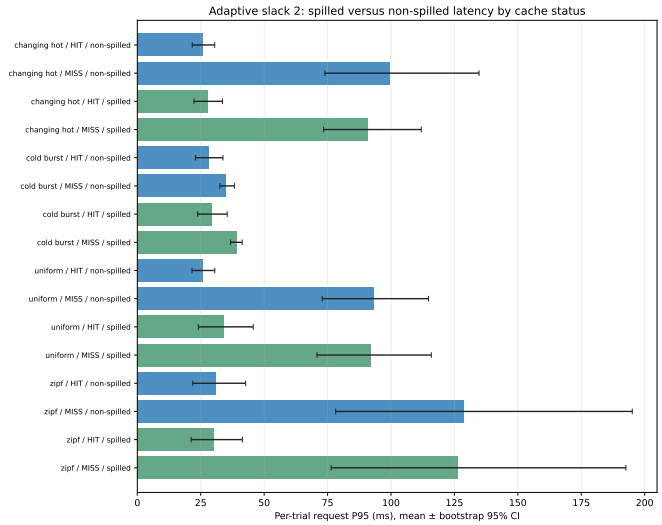


Fig. 5. Per-trial P95 latency for spilled and non-spilled requests, split by workload and HIT/MISS status, at adaptive slack 2. Source: `routing-sweep-2026-10-25T19-23-50-602Z-requests.csv`.

survey on replica server placement algorithms for content delivery networks," *IEEE Communications Surveys & Tutorials*, vol. 19, no. 2, pp. 1002–1026, 2017.

[11] Y. Chen, R. H. Katz, and J. D. Kubiatowicz, "Dynamic replica placement for scalable content delivery," in *Peer-to-Peer Systems*, ser. Lecture Notes in Computer Science. Springer, 2002, pp. 306–318.

[12] A. Archer, K. Aydin, M. H. Bateni, V. Mirrokni, A. Schild, R. Yang, and R. Zhuang, "Cache-aware load balancing of data center applications," *Proceedings of the VLDB Endowment*, vol. 12, no. 6, pp. 709–723, 2019.

[13] A. Vattani, F. Chierichetti, and K. Lowenstein, "Optimal probabilistic cache stampede prevention," *Proceedings of the VLDB Endowment*, vol. 8, no. 8, pp. 886–897, 2015.

[14] R. T. Fielding, M. Nottingham, and J. Reschke, "Http semantics," RFC Editor, RFC 9110, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110.html

[15] *HAProxy Configuration Manual, Version 3.0*, HAProxy Technologies, 2026, version 3.0.30-2, accessed October 26, 2026. [Online]. Available: https://docs.haproxy.org/3.0/configuration.html